FileWorkbench Next

AI 開発引継ぎ・最終目標仕様書

対象ベース: Ver.3.1 RC Module I/F Developer

文書の目的	次の AI または開発者が、設計思想を崩さずに開発を継続できるようにする
現時点の中心	モジュール化、共通 I/F、Developer Mode、単体試験
最終目標	利用者モードと開発者モードを備えた、安全で拡張可能なファイル管理プラットフォーム
更新日	2026-10-07

この文書と最新のソース ZIP を AI へ一緒に渡してください。

1. この文書の使い方

この仕様書は、過去の単一ファイル型アプリから現在のモジュール型アーキテクチャまでの方針を整理し、次の AI が短時間で開発を再開するための引継ぎ資料です。会話履歴がなくても、最新 ZIP と本書だけで現状把握、起動、単体試験、修正、再統合ができる状態を目標にしています。

AI への最初の指示
まず本書を読み、Developer Mode を起動し、全モジュールの契約試験を行ってください。既存機能を削除したり、旧統合版だけへ戻したりせず、共通 I/F 経由で段階的に改善してください。

1.1 AI へ渡すもの

- 最新のソース一式 ZIP: FileWorkbench_Next_3.1_RC_Module_IF_Developer.zip
- 本書: FileWorkbench_AI_Development_Handoff_Specification.docx
- 必要に応じて、画面のスクリーンショット、エラー全文、希望する変更内容

1.2 AI が最初に行う確認

1. ZIP を展開し、README.txt と本書を読む
2. requirements.txt の依存関係を確認する
3. developer_mode.bat を起動する
4. モジュールごとに「契約試験」と「単体ウィンドウ」を確認する
5. start_app.bat で統合画面を確認する
6. 変更前にバックアップまたは Git コミットを作る

2. 製品の最終目標

FileWorkbench Next は、ファイル構成の確認、出力、比較、リネーム、プレビュー、HASH・重複確認を、安全に組み合わせて利用できる Windows 向けデスクトップアプリです。最終形では「利用するアプリ」と「アプリを組み立てて試験する Studio」の二つの顔を持ちます。

モード	主な利用者	目的
User Mode	一般利用者	構成ツリー、出力、比較、リネーム、プレビュー、HASH を分かりやすく利用する
Developer Mode	開発者・AI	モジュールの自己記述、I/F 契約、コンテキスト、イベント、単体 UI、組み込み状態を確認する
将来の Studio Mode	開発者・AI	ModuleBox をキャンバスへ配置し、接続とレイアウトを保存してアプリ構成を生成する

2.1 非交渉の設計方針

- 通常画面と Developer Mode を両方残す
- 各機能は単体起動できるモジュールにする

- メイン側は module.py を直接操作せず、interface.py の共通契約を介する
- module.json をモジュールの自己紹介・発見情報として維持する
- 破壊的なファイル操作は確認、上書き防止、ログ、可能な範囲の Undo を備える
- assets/app.svg、assets/app.ico、assets/app.png は同一の最新版デザインに統一する
- ルート直下は起動に必要なファイルだけに保ち、docs、scripts、tests、legacy へ整理する

3. バージョンの経緯と現在地

段階	主な内容	扱い
2.5 系	構成ツリー、左右比較、リネーム、Undo、HASH、プレビュー、出力	旧統合版の機能基準
2.6 系	プレビュー前後移動、日本語右クリック、カラー UI、アイコン統一	UX 改善
2.7 系	リネーム確認表、可変列、詳細確認、精査チェック	RC 段階。Final 表記は取り消し
3.0 RC Modular	機能を tree / rename / preview / compare / hash へ分離	モジュール化の土台
3.1 RC Module I/F Developer	共通 I/F、ModuleBox、module.json、Developer Mode	現在の開発中心

重要
3.1 RC は完成版ではありません。Developer Mode と I/F 設計を中心に、旧統合版の機能を各モジュールへ正確に移植し直すための基盤です。

4. 現在のフォルダー構成

```
FileWorkbench_Next_3.1_RC_IF_Developer/
├─ app_main.py              # 互換用の薄い起動入口
├─ start_app.bat            # User Mode
├─ developer_mode.bat       # Developer Mode
├─ run_module.bat           # モジュール単体起動
├─ requirements.txt
├─ README.txt
├─ LICENSE.txt
├─ assets/                  # app.svg / app.ico / app.png / 機能アイコン
├─ src/fileworkbench/
│  ├─ app.py                # 統合画面
```

```
│ ├─ module_registry.py        # モジュール登録
│ ├─ core/                     # 共通処理
│ ├─ interfaces/               # 共通 I/F 契約、Adapter、EventBus、ModuleBox
│ ├─ developer/workbench.py    # Developer Mode UI
│ └─ modules/
│    ├─ tree/
│    ├─ rename/
│    ├─ preview/
│    ├─ compare/
│    └─ hash/
│       各フォルダー: module.py / interface.py / module.json
├─ docs/                       # 仕様書、履歴、画像
├─ scripts/                    # ビルド、配布、補助実行
├─ tests/                      # 自動試験
└─ legacy/                     # 旧統合版。移植元として保持
```

4.1 主要ファイルの役割

ファイル	役割	AI が変更するときの注意
module.py	モジュール本体の UI と機能	他モジュールへ直接依存させない
interface.py	共通 I/F へ適合させる Adapter	入力、コマンド、イベントを明示する
module.json	ID、名称、版、アイコン、能力、入力宣言	コードと矛盾させない
contracts.py	ModuleDescriptor / Context / Event / Port	互換性を壊す変更は版を上げる
module_box.py	モジュール UI を箱として統合画面へ配置	タイトル、版、別窓操作を共通化
event_bus.py	イベントを疎結合に中継	イベント名と payload 仕様を記録する
module_registry.py	利用可能なモジュールを登録	将来は module.json 自動検出へ移行
developer/workbench.py	単体・組み込み・契約・Context・イベント試験	AI の主要な診断入口として残す

5. 共通 I/F 仕様

5.1 ModuleDescriptor

モジュールの自己紹介です。Developer Mode の左側に表示され、AI やメインアプリが「何ができるモジュールか」を把握するために使います。

```
ModuleDescriptor(
    module_id: str,
    display_name: str,
    version: str,
    icon: str,
    description: str,
    capabilities: tuple[str, ...],
    accepted_inputs: tuple[str, ...],
    emitted_events: tuple[str, ...]
)
```

5.2 ModuleContext

メインまたは別モジュールから、対象ファイルや作業条件を渡す入力箱です。

```
{
  "selected_paths": ["C:/work/sample.jpg"],
  "working_directory": "C:/work",
  "options": {"mode": "preview"},
  "source_module": "tree"
}
```

5.3 ModuleEvent

モジュールから外部へ通知する出力箱です。ファイル選択、処理完了、エラーなどをイベントとして通知します。

```
ModuleEvent(
    name="selection.changed",
    source="tree",
    payload={"selected_paths": ["C:/work/sample.jpg"]}
)
```

5.4 ModulePort の必須操作

操作	意味	期待動作
create_widget(parent)	UI を取得する	埋め込み可能な QWidget を返す
receive_context(context)	外部入力を受け取る	UI と内部状態を更新し、受信イベントを出す
current_context()	現在の入力を返す	直近の ModuleContext を返す

操作	意味	期待動作
execute(command, payload)	共通コマンドを実行する	未対応コマンドは明確なエラーにする
subscribe(callback)	イベント購読を登録する	ModuleEvent を callback へ渡す
shutdown()	終了処理	UI、スレッド、ファイル、メディアを安全に解放する

6. Developer Mode 仕様

Developer Mode は削除しません。AI がコード一式を受け取ったときに、ソース全体を読み切る前でも、モジュール一覧、I/F、単体 UI、Context 受信、イベント、契約適合を確認できる診断画面です。

領域	現在の役割	将来の改善
Module List	tree / rename / preview / compare / hash を選択	module.json の自動検出、状態・依存関係表示
Inspector	Descriptor を JSON 表示	入力・出力・コマンドをタブ表示
組み込み試験	ModuleBox として右側へ配置	複数ボックス、移動、リサイズ、削除
単体ウィンドウ	選んだモジュールだけ起動	テストデータセット選択と結果保存
契約試験	必須プロパティと Widget を確認	command、event、型、例外、shutdown まで自動試験
Context Editor	JSON を ModuleContext として送信	入力テンプレート、履歴、バリデーション
Event Log	受信・完了イベントを表示	時刻、色分け、検索、保存、フロー図

6.1 Developer Mode の起動と基本試験

7. requirements.txt の依存関係を導入する
8. developer_mode.bat を起動する
9. モジュールを選び、契約試験を押す
10. 単体ウィンドウで基本操作を試す
11. 組み込み試験で ModuleBox として表示する
12. Context JSON を送信し、ログへ context.received が出ることを確認する
13. 起動失敗したモジュールがあっても他モジュールを試験できる設計へ改善する

7. 各モジュールの責務と到達目標

ID	責務	現在の基礎	最終到達目標
tree	構成ツリーと出力	フォルダー表示、検索、TXT/CSV/Excel/HTML	D&D、再スキャン、選択イベント、固定出力バー、安全な保存
rename	安全な一括リネーム	置換、Prefix/Suffix、可変列、詳細確認	連番、日付、正規表現、差分強調、Dry Run、Undo、履歴
preview	内容確認と整理	画像、前後移動、コピー/移動/ごみ箱/HASH	画像・テキスト・PDF・動画・音声、ズーム、パン、等倍、サムネイル
compare	2 フォルダー比較	一致/不一致/片側、検索、差分のみ	バックグラウンド処理、詳細差分、フィルター、同期表示
hash	HASH 台帳と重複	SHA256、全件/重複、検索、CSV	進捗、速度、ETA、キャンセル、失敗一覧、重複グループ操作

7.1 旧統合版から維持すべき機能

- 構成ツリーの 4 形式出力と安全保存
- 一括リネームの禁止文字・予約名・衝突検査・上書き防止
- 再起動後を含む Undo 履歴の考え方
- HASH のバックグラウンド計算、進捗、キャンセル、CSV
- プレビューの前後移動、画像フィット、日本語右クリック、メディア再生
- 左右比較の一致・相違・片側判定
- 切り離し表示とドラッグ＆ドロップ
- operations.csv、rename_history.json などの操作履歴

8. 安全性要件

最優先
リネーム、移動、削除など、利用者のデータを変更する処理は、便利さより安全性を優先します。

- 既存ファイルを無断で上書きしない
- 禁止文字、予約名、末尾空白・ピリオド、パス長、同名衝突を事前検査する
- 実行前に変更前と変更後を一覧表示する
- 破壊的操作は件数と対象を示して確認する
- 削除は可能な限りごみ箱を使用する
- 操作結果と失敗理由をログに残す
- 処理途中で失敗した場合の復元方針を用意する
- 外部で変更されたファイルを Undo しない
- UI が固まらないよう重い処理はワーカーで実行する

- モジュール終了時にスレッド、メディアソース、ファイルハンドルを解放する

9. 依存関係と実行環境

項目	方針
OS	Windows 11 64bit を主要対象
Python	Python 3.11 から 3.13 を想定。使用中の Python と pip を一致させる
UI	PySide6
主な依存	send2trash、openpyxl、必要に応じ PyInstaller ほか requirements.txt 記載分
導入	py -3.13 -m pip install -r requirements.txt のように対象 Python を明示
推奨環境	プロジェクト専用 venv を作り、グローバル環境の壊れたパッケージ影響を避ける

既知の環境注意
send2trash が未導入だと preview モジュールの import で Developer Mode 全体が起動できませんでした。最終設計では、モジュールの遅延読込と個別エラー隔離を実装し、1 モジュールの依存不足で他モジュールを停止させないこと。

10. AI が守る開発ワークフロー

14. 最新 ZIP を新しい作業フォルダーへ展開する
15. 本書、README、module.json、interface.py、Developer Mode を確認する
16. 対象モジュールの契約試験と単体起動を変更前に実施する
17. 変更範囲を対象モジュール、I/F、共通部に限定する
18. 新しい入力、コマンド、イベントは module.json と仕様書へ追記する
19. 自動テスト、構文チェック、単体 UI、組み込み UI、通常画面を確認する
20. ファイル操作機能は一時フォルダーのテストデータで試験する
21. 変更履歴、受入チェック、依存関係、版数を更新する
22. ルート直下を散らかさず、所定フォルダーへ配置する
23. 完成物はパッチだけでなく、適用済みのソース一式 ZIP として渡す

10.1 変更完了の条件

分類	完了条件
コード	構文エラーなし。未使用 import、直接依存、ハードコードされた絶対パスを残さない
I/F	Descriptor、Context、Event、Command がコードと宣言で一致

分類	完了条件
単体	Developer Mode から契約試験と単体 UI が動く
統合	ModuleBox へ組み込み、通常画面からも起動できる
安全	確認、上書き防止、エラー表示、ログ、復元方針を確認
配布	requirements、README、仕様書、履歴、ZIP 内容と版数が一致

11. 推奨ロードマップ

優先	作業	目的
P0	モジュール遅延読込と依存エラー隔離	一つの依存不足で Developer Mode 全体を失敗させない
P0	Developer Mode の契約試験拡張	Context、Command、Event、shutdown も自動確認
P1	旧統合版と各モジュールの機能差分表を作る	移植漏れを可視化する
P1	tree selection.changed から preview への Context 連携	最初の実用的モジュール間通信
P1	レイアウトキャンバスと複数 ModuleBox	Studio 化の中心
P1	レイアウト JSON の保存と再読込	用途別アプリ構成を再現する
P2	イベントモニターの時刻・色・検索・保存	通信を初心者と AI に見える化
P2	module.json 自動検出	プラグイン追加時のメイン変更を不要にする
P2	各モジュール内 tests と fixtures	AI が安全に回帰試験できるようにする
P3	EXIF、OCR、PDF、Archive などの追加モジュール	プラットフォーム拡張

12. 最終形の通信イメージ

```
[User Mode / Studio Canvas]
        |
```

```
        v
    [ModuleBox]
        |
     ModulePort
        |
 +-------+--------+
 |       |        |
 Tree    Preview  Rename
 |       |        |
 +---- EventBus --+
        |
    ModuleEvent
        |
 Developer Monitor / Context Router
```

例:

Tree: selection.changed

-> Context Router

-> Preview.receive_context(selected_paths)

-> Preview: context.received

-> Event Monitor に記録

13. AI へ渡す実行指示テンプレート

そのままコピーして使えます

以下の文章の後ろへ、今回実装してほしい内容を追加してください。

あなたは FileWorkbench Next の継続開発担当です。

添付した最新ソース ZIP と「AI 開発引継ぎ・最終目標仕様書」を最初に読んでください。

必須方針:

1. Developer Mode、共通 I/F、module.json、各 interface.py を維持する。
2. 機能は該当モジュール内へ実装し、他モジュールへ直接依存させない。
3. モジュール間連携は ModuleContext、ModuleEvent、EventBus を使う。
4. 破壊的なファイル操作は確認、上書き防止、ログ、復元方針を持たせる。
5. 変更後は契約試験、単体 UI、組み込み UI、通常画面を確認する。
6. requirements、module.json、README、履歴、受入チェックをコードと一致させる。
7. パッチだけでなく、修正を適用したコード一式 ZIP を出力する。
8. assets/app.svg、assets/app.ico、assets/app.png を同じ最新版デザインに保つ。

今回の作業:

[ここへ依頼内容を書く]

成果物:

- 修正済みソース一式 ZIP
- 変更点一覧
- 試験結果
- 残課題と次の推奨作業

14. AI 向け最終チェックリスト

☐ 最新ソースと仕様書を読んだ

☐ Developer Mode が起動した

☐ 全モジュールを列挙できた

☐ 対象モジュールの契約試験を実施した

☐ module.json と interface.py を更新した

☐ 単体ウィンドウで試した

☐ ModuleBox へ組み込んだ

☐ Context 送信と Event ログを確認した

☐ User Mode を確認した

☐ 安全性要件を確認した

☐ requirements を更新した

☐ README と変更履歴を更新した

☐ 版数とファイル名を統一した

☐ ZIP の内容と起動方法を確認した

☐ 残課題を明記した

15. 現時点での結論

今後の開発中心は、巨大な app_main.py ではなく、モジュール本体、共通 I/F、ModuleBox、EventBus、Developer Mode です。User Mode は利用者向けの完成画面、Developer Mode は人間と AI が構造を理解し、安全に単体・組み込み試験を行うための開発基盤として、両方を継続します。

最終目標
FileWorkbench Next を、個別機能を安全に追加・試験・接続・配置できる「ファイル管理アプリ + 開発 Studio」に成長させること。

16. 最新版の変更履歴

本章は、AI へ引き渡す時点の最新版までの変更をまとめたものです。版数、構成、実装状態を判断するときは、この章と最新ソース ZIP の内容を照合してください。

版	主な変更	状態
Ver.2.5 系	構成ツリー、4 形式出力、左右比較、一括リネーム、Undo、HASH 台帳、プレビュー、メディア再生、D&D、ログを統合版へ実装	旧統合版。機能移植元として legacy へ保持
Ver.2.6.0	プレビューの前後移動、件数表示、左右キー操作、画像の自動フィットを追加	3.x の preview へ段階移植する
Ver.2.6.1	左メニューと上部ステップのカラー化、app.svg / app.ico / app.png のデザイン統一、日本語右クリック操作を追加	アイコン 3 形式は今後も同一デザインを維持
Ver.2.7.0	一度 Final 表記にしたが、リネーム確認表と出力配置などに未完了項目があり Final を取り消し	正式完成版として扱わない
Ver.2.7.1 RC	リネーム確認表の可変列、列幅保持、フルパスツールチップ、ダブルクリック詳細確認、下部固定アクションバーを追加	モジュール化前の改善版
Ver.3.0 RC Modular	tree / rename / preview / compare / hash を独立モジュールへ分離。統合起動と単体起動を用意	モジュール化の第一段階
Ver.3.0 RC Modular Clean	src/fileworkbench 配下へ整理。core、modules、docs、scripts、tests、legacy へ分割し、ルートを簡潔化	現在のフォルダー構成の基礎
Ver.3.1 RC Module I/F Developer	ModuleDescriptor、ModuleContext、ModuleEvent、ModulePort、Adapter、EventBus、ModuleBox、module.json、interface.py、Developer Mode を導入	現在の開発ベース
Ver.3.1 RC 動作確認	send2trash 不足を導入後、Developer Mode 起動、構成ツリーの組み込み、Context 送信、context.received イベントを確認	I/F 基盤の実動作を確認済み

16.1 現在確認できている動作

• Developer Mode が起動し、tree / rename / preview / compare / hash の一覧を表示する。

• 選択したモジュールの ModuleDescriptor を JSON 形式で確認できる。

• 構成ツリーモジュールを ModuleBox として Developer Mode 右側へ組み込める。

• ModuleContext を送信すると、context.received イベントと送信成功ログを表示する。

• モジュール単体ウィンドウ、組み込み試験、契約試験の入口を備える。

16.2 現時点の既知課題

• preview の send2trash など、1 モジュールの依存不足が Developer Mode 全体の import 失敗につながる。遅延読込と個別エラー隔離が必要。

• 3.x の各モジュールは、旧統合版の全機能をまだ完全移植していない。機能差分表を作り、漏れなく移植する必要がある。

• EventBus は基盤があるが、tree の selection.changed から preview への自動 Context 連携は未完成。

• ModuleBox の複数配置、移動、リサイズ、削除、レイアウト保存は未実装。

• module.json は存在するが、module_registry.py はまだ静的登録が中心。将来は自動検出へ移行する。

• 各モジュールの自動試験、テスト用 fixtures、破壊的操作の回帰試験を整備する必要がある。

17. AI 向け依頼文 完成版

以下を、最新ソース ZIP と本仕様書を添付したうえで、そのまま AI へ渡してください。「今回の実装依頼」の部分だけ、目的に合わせて書き換えます。

あなたは「FileWorkbench Next」の継続開発担当です。

添付した次の 2 ファイルを最初に確認してください。

1. 最新の修正済みソース一式 ZIP
2. FileWorkbench_AI_Development_Handoff_Specification_Final.docx

最初に行うこと:

- 仕様書、README.txt、requirements.txt を読む。
- ZIP を展開し、現在のフォルダー構成と版数を確認する。
- developer_mode.bat を起動する。
- 全モジュールを列挙し、対象モジュールの契約試験、単体ウィンドウ、組み込み試験を実施する。
- 変更前の状態と既知課題を整理してから実装を開始する。

絶対に維持する設計:

- User Mode と Developer Mode の二階建て構成。
- src/fileworkbench/modules/<module>/ の module.py、interface.py、module.json。
- ModuleDescriptor、ModuleContext、ModuleEvent、ModulePort、ModuleBox、EventBus。
- モジュール単体起動と Developer Mode での単体・組み込み・契約試験。
- assets/app.svg、assets/app.ico、assets/app.png の同一最新版デザイン。
- docs、scripts、tests、legacy による整理されたフォルダー構成。

実装規則:

1. 新機能は原則として該当モジュール内へ実装する。
2. 他モジュールの module.py を直接呼び出さない。
3. モジュール間連携は ModuleContext、ModuleEvent、EventBus を利用する。
4. I/F を変更した場合は interface.py、module.json、Descriptor、仕様書を同時に更新する。
5. 1 モジュールの依存不足や import 失敗で Developer Mode 全体を停止させない。遅延読込と個別エラー表示を優先する。
6. リネーム、移動、削除などの破壊的操作には、事前確認、衝突検査、上書き防止、ログ、失敗時の復元方針を持たせる。
7. 重い処理はバックグラウンドで実行し、進捗、キャンセル、エラーを UI へ表示する。
8. 旧統合版 legacy は移植元として参照し、既存機能を理由なく削除しない。
9. ルート直下へ作業ファイルや一時ファイルを増やさない。
10. パッチだけではなく、変更適用済みのコード一式 ZIP を完成成果物として返す。

今回の実装依頼:

[ここに今回実装してほしい機能・修正内容を書く]

必須試験:

- Python 構文確認。

- 対象モジュールの契約試験。
- 対象モジュールの単体ウィンドウ試験。
- Developer Mode への ModuleBox 組み込み試験。
- ModuleContext 受信と ModuleEvent 出力の確認。
- User Mode の起動確認。
- ファイル操作はテスト用一時フォルダーで実施。
- ZIP 展開試験と依存関係確認。

更新対象:
- 対象コード。
- module.json と interface.py。
- requirements.txt。
- README.txt。
- CHANGELOG または最新版変更履歴。
- 受入チェックリストと試験結果。
- 必要な仕様書。

成果物:
1. 修正適用済みソース一式 ZIP。
2. 変更点一覧。
3. 実施した試験と結果。
4. 未実装項目、既知課題、次の推奨作業。
5. 起動方法と、利用者が確認すべきポイント。

禁止事項:
- Developer Mode や共通 I/F を削除して、巨大な単一 app_main.py へ戻さない。
- 動作未確認の状態で Final と表記しない。
- ソース、仕様書、ビルドスクリプト、同梱ファイルの版数を不一致にしない。
- 利用者の実データを使って破壊的試験をしない。
- エラーを握りつぶしたり、未確認内容を「確認済み」と記載したりしない。

17.1 短い依頼例

今回の実装依頼:
- tree の selection.changed を EventBus へ発行する。
- Context Router を追加し、選択ファイルを preview.receive_context へ渡す。
- Developer Mode の Event Log に時刻、送信元、イベント名、宛先を表示する。
- 既存の tree、preview 単体動作を維持する。
- 修正済みコード一式 ZIP を返す。